

PRODUCTION FAILURE ANALYSIS

**Your classical ML model scored
95% in k-fold CV.**

No leaks. No shortcuts. Then production hit **60%**.

What went wrong?

The Assumption That Breaks

K-fold assumes train and test sets come from the exact same world.

🕒 **Time shift:** Train on Jan–Mar. Deploy in April.

📱 **Source shift:** Train on iOS. Test set is Android.

K-fold only shuffles within train. It never sees this distribution gap.



Covariate Shift Validation Gap

The Hard Way

100+ features. Traditional drift detection won't save you.

- ✗ **KS-test:** One per feature. That's 100+ individual runs.
- ✗ **PSI:** Per-feature stability index. Threshold is arbitrary.
- ✗ **PCA:** Can't tell you which features matter to your model.

None of these tell you if the drift affects your predictions.

Feature Drift Overhead

feature_01 (KS-Test)



feature_02 (PSI Threshold)



feature_03 ... feature_100+



Adversarial Validation

Train a model to separate your train set from your test set.

1. Label train rows → **0**, test rows → **1**
2. Combine. Train a binary classifier (LightGBM/XGBoost).
3. Read the AUC.

AUC $\approx$ 0.5 → distributions match. CV is safe.

AUC $>$ 0.7 → severe shift. Your CV is lying.

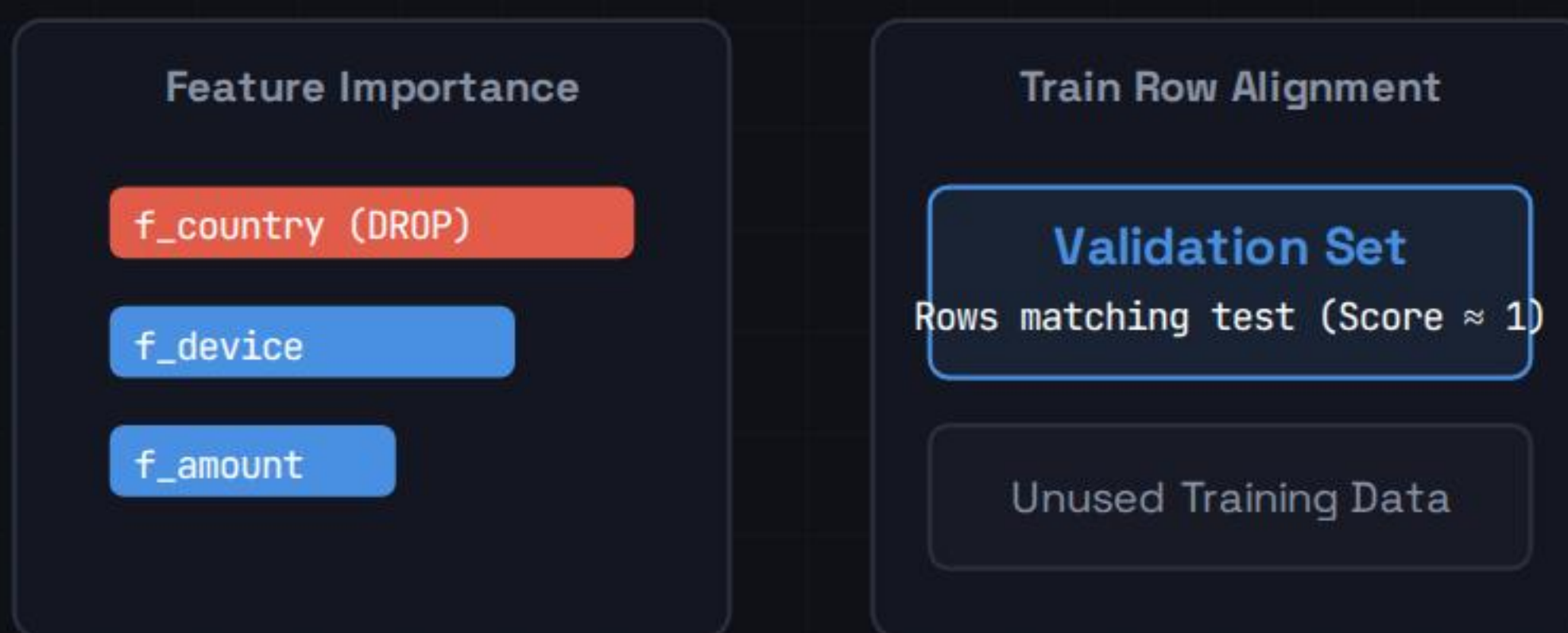


What You Do With the Fix

A high AUC doesn't just flag the problem—it hands you the fix.

- ✂ **Drifting features:** Check feature importance. The top-ranked variable is your culprit. Drop it.
- ▼ **Better validation:** Isolate train rows scored closest to 1. They mirror your test set. Use them as your validation fold.

One run. All features. Interaction-aware.



Automate in Production

Set a cron job. Let the AUC tell you when data has shifted.

🔄 After every pipeline run, retrain adversarial model on fresh prod data.

⚠️ If **AUC > 0.7**, trigger alerts immediately.

No thresholds to tune. Your model stops walking into a world it hasn't seen.

